LEARNING GUIDE

From QA to DevOps

Every AWS and DevOps service used in the Secure Employee Management API project, explained in plain language, with QA and performance-testing comparisons

Written for	**Akash Ajay** – QA engineer (5 years, performance testing) moving into a DevOps role
Based on	The real system you built: CloudFront → AppSync → ALB → ECS Fargate → RDS, with GitHub Actions CI/CD
How to use it	Read Part A once for the big picture. Use Part B as a reference per service. Do the exercises in Part C on your own setup – they are the fastest way to make it stick.
Date	14 October 2026

Contents

Part A – The big picture

1 The mindset shift: QA → DevOps

2 Your skills already map to DevOps

3 One request, end to end

4 AWS foundations (Region, AZ, ARN, IAM)

Part B – Every service explained

5 Docker

6 Amazon ECR

7 Amazon VPC (networking)

8 Security groups

9 Application Load Balancer

10 Amazon ECS

11 AWS Fargate

12 Amazon RDS

13 AWS Secrets Manager (and KMS)

14 AWS AppSync & GraphQL

15 Amazon CloudFront

16 IAM roles, STS & OIDC

17 Amazon CloudWatch

18 Amazon SNS

19 AWS CloudTrail

20 GitHub Actions (CI/CD)

21 Trivy & ECR scanning

22 Terraform (Infrastructure as Code)

23 Cost management

Part C – Practise and grow

24 Hands-on exercises on your setup

25 Your performance-testing edge

26 Glossary

27 Learning path: next 90 days

1 The mindset shift: QA → DevOps

As a QA engineer you were mostly **verifying** a system someone else built and deployed. As a DevOps engineer you **build, deploy and run** the system – and you are on the hook when it breaks at 2 a.m. The good news: the habits that made you a good tester are exactly the habits that make a reliable DevOps engineer.

As a QA engineer you asked…	As a DevOps engineer you ask…
"Does this feature work?"	"Does it keep working when a server dies, the database restarts, or traffic doubles?"
"Did the test pass on the test environment?"	"Is the environment itself reproducible, versioned and identical to production?"
"What is the response time under 500 users?"	"What do we do automatically when response time degrades – scale, alert, roll back?"
"I found a bug – log a ticket."	"Can the pipeline catch this class of bug automatically, every time, before release?"
"Who deployed this build?"	"Every deployment is automated, traceable to a commit and reversible."

The four ideas that sit underneath everything

1. Automate everything repeatable

Builds, tests, scans, deployments, even infrastructure creation. If a human does it twice, script it. Manual steps are where mistakes happen – you saw this yourself with the `DB_PORT5432` typo.

2. Treat infrastructure like code

Servers, networks and permissions are described in files, reviewed, versioned in Git and recreated on demand – no "snowflake" environments.

3. Design for failure

Things *will* fail. Run more than one copy, check health constantly, replace broken parts automatically, and deploy without downtime.

4. Measure, observe, alert

Logs, metrics and alarms tell you what the system is doing in production. You can't fix what you can't see.

> **The short version**
>
> QA tests the product. DevOps builds the machine that builds, tests, ships and runs the product – and keeps watching it afterwards. Testing doesn't disappear; it moves earlier (into the pipeline) and later (into production monitoring).

2 Your skills already map to DevOps

You are not starting from zero. Most of what you know has a direct DevOps equivalent:

Your QA / performance skill	DevOps equivalent	Where you used it in this project
Regression suite	CI pipeline quality gate	Unit tests run on every push; failure blocks the deployment
Test environment setup	Infrastructure as Code	VPC, subnets, security groups – ideally in Terraform
Test data management	Database seeding / migrations	App creates the `employees` table and seed rows on startup
Load profile, ramp-up	Capacity planning, auto scaling	Sizing tasks at 0.25 vCPU; CPU > 80% alarm
Percentiles (p90/p95/p99)	SLOs, latency monitoring	Dashboard shows ALB response time p50/p95/p99
Error rate in a test run	5xx alarms, error budgets	`employee-5xx-errors` alarm
Monitoring server resources during a test	Observability (metrics, logs)	CloudWatch dashboard: CPU, memory, requests
Warm-up period excluded from results	Health check grace period	60 s grace period on the ECS service
Synthetic monitoring / health probes	Load balancer health checks	`/health` checked every 15 s by the ALB
Negative / failure testing	Chaos and resilience testing	Stopping the DB found a crash bug; stopping a task shows self-healing
Root-cause analysis of a failed test	Incident troubleshooting	Tracing a 5xx hop by hop: CloudFront → AppSync → ALB → ECS → RDS
Security / vulnerability testing	DevSecOps scanning	Trivy and ECR scan-on-push blocked real CVEs
Test reports and evidence	Pipeline logs, dashboards	Green GitHub Actions run, CloudWatch screenshots

> **How to say it in an interview**
> "Performance testing taught me to think about systems under stress – throughput, latency percentiles, saturation and failure modes. In DevOps I use the same thinking to design health checks, alarms, scaling and safe deployments, and I automate the tests I used to run by hand."

3 One request, end to end

Before looking at each service, follow a single request through the system you built. Each numbered step is a service explained in Part B.

1 Client (Postman)
POST https://d1a7kv1va3r66w.cloudfront.net/graphql · header x-api-key · body { employees { id name } }

2 CloudFront (edge)
Terminates HTTPS near the user, does not cache (GraphQL POST), forwards headers except Host to AppSync

3 AppSync (GraphQL API)
Checks the API key (401 if wrong), validates the query against the schema, runs the resolver → GET /employees on the ALB

4 Application Load Balancer
Picks one of the healthy targets in emp-tg (round robin) and forwards to its private IP on port 8080

5 ECS Fargate task (your Docker container)
Express app in a private subnet runs the route handler; credentials came from Secrets Manager at startup

6 RDS PostgreSQL
SELECT id, name, department, email FROM employees (over TLS, allowed only from the ECS security group)

7 The answer travels back the same way
App → ALB → AppSync (parses JSON, returns only requested fields) → CloudFront → client. Every hop writes metrics/logs to CloudWatch.

A single GraphQL request through the system

> **Why this matters for troubleshooting**
> When something fails, you test each hop on its own, from the outside in. The first hop that fails is where the problem lives. This is the same as isolating a slow transaction in a load test by measuring each tier separately.

4 AWS foundations

Concept	Plain meaning	In your project
Region	A geographic area with its own independent set of AWS data centres. Resources live in one region.	`us-east-1` (N. Virginia)
Availability Zone (AZ)	One or more physically separate data centres inside a region. Spreading across AZs survives a data-centre failure.	`us-east-1a` and `us-east-1b`
ARN	Amazon Resource Name – a unique "full address" for any AWS resource, used in permissions and references.	`arn:aws:iam::324440884610:role/ecsTaskExecutionRole`
IAM	Identity and Access Management – who (users, roles, services) can do what (actions) to which resources.	Execution role, GitHub Actions role, policies
Policy	A JSON document of Allow/Deny rules: *Action*, *Resource*, optional *Condition*.	`read-employee-db-secret`, `ecr-push-ecs-deploy`
Role	An identity with permissions but no password; it is *assumed* temporarily by a service or person.	ECS assumes the execution role; GitHub assumes the CI role
Least privilege	Give only the exact permissions needed – nothing more.	Execution role can read exactly one secret
Console vs CLI vs IaC	Three ways to change AWS: click in the web console, run `aws` commands, or describe it in code (Terraform).	You used console + CLI; Terraform is the next step

QA ANALOGY

IAM is like test-account permissions in an application under test: a "read-only auditor" account must not be able to delete records. You'd write negative tests for that. In AWS, the **IAM policy simulator** is your negative-test tool – you used it to prove the execution role was *allowed* one secret and *denied* another.

5 Docker

Containers

Packaging

Docker packages your application **together with everything it needs to run** – the Node.js runtime, libraries, files – into an **image**. A running copy of an image is a **container**. The same image runs identically on your laptop, in CI and in AWS.

QA ANALOGY

"It works on my machine" is a classic tester complaint. A Docker image is a **frozen, versioned test environment** – the exact build you tested is the exact build that goes to production.

IN YOUR PROJECT

Your `Dockerfile` starts from `node:22-alpine`, applies OS patches, installs only production dependencies, removes npm, adds the RDS certificate bundle, switches to the non-root `node` user, exposes port 8080 and defines a health check.

Key terms

Term	Meaning
Image	Read-only template (like a class). Built from a Dockerfile.
Container	A running instance of an image (like an object).
Dockerfile	Recipe of instructions (`FROM` , `COPY` , `RUN` , `USER` , `CMD`) that builds the image.
Layer	Each instruction creates a cached layer. Copying `package.json` before the source means dependencies are reinstalled only when they change → faster builds.
Base image	The starting point (`node:22-alpine`). Alpine is a tiny Linux, so fewer packages and fewer vulnerabilities.
Tag	A version label, e.g. `employee-api:1.2.0` .
.dockerignore	Files kept out of the build (like `.gitignore`): `node_modules` , tests, `.env` .
PID 1 / exec form	`CMD ["node","src/server.js"]` makes Node the main process so it receives the stop signal (SIGTERM) and can shut down gracefully.

What goes wrong

- Running as root (security risk)
- Huge images with build tools and secrets inside
- App ignores SIGTERM → requests cut off during deploys
- Old base image with known CVEs

How to test it

- `docker exec <c> whoami` → must not be root
- `docker stop` → logs show graceful shutdown, exit code 0
- Scan the image with Trivy
- Run the container and hit `/health`

Interview one-liner: "A Docker image is an immutable, versioned package of the app and its runtime; the same artefact is tested in CI and deployed to production, which removes environment drift."

6 Amazon ECR

Elastic Container Registry

Image storage

ECR is a **private, secure storage place for Docker images** in your AWS account. You push images to it; ECS pulls from it when starting tasks.

QA ANALOGY

It's the **build artefact repository** – like the shared folder or Nexus/Artifactory where testers fetched "build 1.2.0" to test. Each tag is a specific build you can always get back.

IN YOUR PROJECT

Repository `employee-api`. Manual releases used `1.0`, `1.1.0`, `1.2.0`. The pipeline tags images with the **git commit SHA**, so any running container can be traced to its exact code. Scan-on-push checks every image for known vulnerabilities.

Why not use the `latest` tag?

`latest` moves every time someone pushes. If ECS restarts a task tomorrow, it might pull different code than the task running next to it – two versions in production with no record of why. Immutable tags (versions or commit SHAs) make deployments reproducible and rollback trivial.

What goes wrong

- `CannotPullContainerError` – tag doesn't exist, or task has no network route to ECR
- Login token expired (12 hours)
- Pushing an image index that the basic scanner can't scan

How to check it

- ECR console → image tags and push time
- Vulnerabilities tab after scan-on-push
- `aws ecr describe-images`

Interview one-liner: "ECR stores our images privately; CI pushes an image tagged with the commit SHA, ECR scans it on push, and ECS pulls that exact tag – so every deployment is traceable and reversible."

7 Amazon VPC (networking)

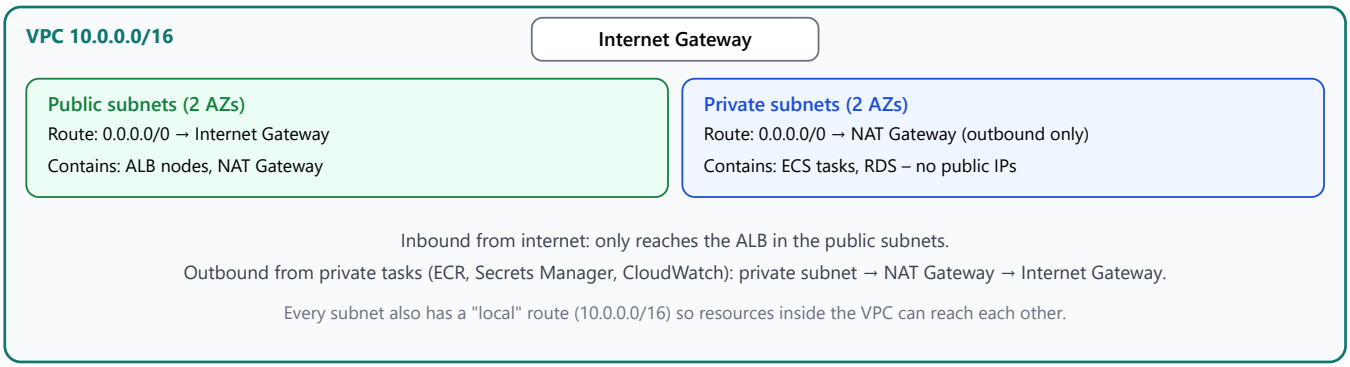
Virtual Private Cloud

Network

A VPC is **your own private network inside AWS**. You decide its IP range, divide it into subnets, and control exactly what can talk to the internet.

QA ANALOGY

Think of an office building. The **VPC** is the building. **Public subnets** are the reception area – visitors can walk in. **Private subnets** are the back offices – no outside door; staff can go out (through the **NAT Gateway**, like a staff exit) but visitors can't come in. The **Internet Gateway** is the main entrance.



Key terms

Term	Meaning	Your setup
CIDR	A way of writing an IP range. `/16` = 65,536 addresses, `/20` = 4,096.	VPC 10.0.0.0/16, subnets /20
Subnet	A slice of the VPC that lives in one AZ.	2 public + 2 private
Route table	Rules saying where traffic for each destination goes. What makes a subnet "public" is a route to the Internet Gateway.	public → IGW, private → NAT
Internet Gateway	Two-way door between the VPC and the internet.	`employee-igw`
NAT Gateway	Lets private resources start connections *out*, but nobody can connect *in*.	One, in public subnet 1
VPC endpoint	Private connection to an AWS service without going through the internet/NAT.	Not used (future improvement)

What goes wrong

- Private tasks with no NAT route → can't pull images or secrets
- ALB placed in private subnets → unreachable
- Database in public subnets with public access → exposed
- Single NAT = single-AZ dependency

How to test it

- VPC → Resource map shows subnets, routes, gateways
- Try to reach a task's private IP from outside → must fail
- Check task has no public IP

Interview one-liner: "Only the ALB sits in public subnets; ECS and RDS sit in private subnets with no public IPs. A NAT Gateway gives the tasks outbound-only access to ECR, Secrets Manager and CloudWatch."

8 Security groups

Virtual firewalls

Network security

A security group is a **firewall attached to a resource** (load balancer, task, database). It lists which traffic is allowed **in**; everything else is blocked. Return traffic is allowed automatically ("stateful").

QA ANALOGY

Like access badges: the ALB badge opens the front door to everyone; the ECS badge opens only for people carrying an ALB badge; the database badge opens only for people carrying an ECS badge. A visitor from the street can never reach the database room.

IN YOUR PROJECT

Group	Allows in	From
`emp-alb-sg`	TCP 80	anywhere (0.0.0.0/0)
`emp-ecs-sg`	TCP 8080	only `emp-alb-sg`
`emp-rds-sg`	TCP 5432	only `emp-ecs-sg`

Why reference another security group instead of an IP? The ALB's IPs and the tasks' IPs change constantly. "Allow from the ALB's security group" stays correct no matter which IPs they get.

What goes wrong

- ALB targets "unhealthy" because ECS SG doesn't allow 8080 from the ALB SG
- App can't connect to DB (`ETIMEDOUT`) because RDS SG doesn't allow the ECS SG
- Using the default SG for everything (too open)

How to test it

- Curl a task's IP directly → must time out
- Remove a rule (in a test) → watch targets go unhealthy (Exercise 3)
- Look for any 0.0.0.0/0 on app or DB ports

Interview one-liner: "Security groups are chained: internet → ALB on 80, ALB → ECS on 8080, ECS → RDS on 5432, using group references so each layer only accepts the layer in front of it."

9 Application Load Balancer

ALB + target group

Traffic distribution

The ALB is the **single front door** that receives HTTP requests and **spreads them across healthy copies** of your app. It constantly health-checks each copy and stops sending traffic to broken ones.

QA / PERFORMANCE ANALOGY

You've watched load balancers during load tests – requests spread across servers, one slow node drags p99 up. The **health check** is a synthetic monitor: a tiny automated test (`GET /health` every 15 s) that decides whether a server is "in" or "out".

IN YOUR PROJECT

`emp-alb` listens on HTTP 80 and forwards to target group `emp-tg` (port 8080). Healthy after 2 passes, unhealthy after 2 failures, every 15 s. Deregistration delay 30 s. ECS registers task IPs automatically.

Key terms

Term	Meaning
Listener	The port/protocol the ALB listens on (HTTP:80) and what to do with requests (forward to a target group).
Target group	The pool of backends plus the health-check settings. Fargate needs target type **IP**.
Health check	Periodic request to `/health`; thresholds decide healthy/unhealthy.
Deregistration delay	"Connection draining": how long the ALB lets in-flight requests finish on a target being removed.

Reading ALB errors (memorise this)

Code	Meaning	First thing to check
502	Target sent an invalid response or closed the connection	App crashing/restarting; app logs
503	No healthy targets	Target group → Targets tab; security groups; is the service running?
504	Target didn't respond in time	Slow app/DB; timeouts

Interview one-liner: "The ALB health-checks each task on /health and only routes to healthy ones; 503 means no healthy targets, 502 a bad response from the app, 504 a timeout."

10 Amazon ECS

Elastic Container Service

Container orchestration

ECS is the **manager** that runs your containers, keeps the right number running, replaces failed ones, connects them to the load balancer and performs deployments.

QA ANALOGY

A **task definition** is a **test plan** (what to run, with what settings). A **task** is one **execution** of that plan. A **service** is a scheduler that says "always keep 2 executions running – if one dies, start another". The **cluster** is the folder that groups them.

Concept	Meaning	Your setup
Cluster	Logical group of services and tasks.	`employee-cluster`
Task definition	Versioned, immutable blueprint: image, CPU/memory, port, env vars, secrets, health check, logs, roles. Each change = new revision.	`employee-task` revisions 1 → 5
Task	A running instance of a task definition (one or more containers).	2 running tasks
Service	Keeps the desired count running, registers with the ALB, does rolling deployments.	`employee-api-service`, desired 2

How ECS keeps the service healthy

1. **Container health check** (inside the task): `wget /health` fails 3 times → ECS stops the task.
2. **ALB health check** (from outside): fails 2 times → ALB stops sending traffic → ECS replaces the task.
3. The service sees *running* < *desired* and starts a new task automatically.
4. The **grace period** (60 s) stops a slow-starting task from being killed while it warms up.

Rolling deployment (zero downtime)

With **minimum 100% / maximum 200%**, ECS starts the new tasks first, waits until they pass health checks, then drains and stops the old ones. The **circuit breaker** automatically rolls back if the new version never becomes healthy.

PERFORMANCE ANGLE – YOU MEASURED THIS

During a rolling deploy you sent 200 requests in a loop: **200 × HTTP 200, 0 errors**. That is a deployment-time availability test – a test most teams never run.

What goes wrong

- Tasks keep stopping → read the **Stopped reason**
- Secret can't be fetched → `ResourceInitializationError`
- Task definition typo (e.g. env var key)
- Desired count 1 → no redundancy

How to test it

- Stop one task → watch a replacement start (Exercise 1)
- Deploy a change while probing → count errors
- Deploy a broken image → watch circuit breaker roll back

Interview one-liner: "ECS is the orchestrator: the task definition is the versioned blueprint, the service keeps the desired number of healthy tasks behind the ALB, replaces failed ones and does rolling deployments with automatic rollback."

11 AWS Fargate

Serverless compute for containers

Compute

Fargate is **where** your containers physically run – without you managing any servers. You say "this task needs 0.25 vCPU and 512 MB" and AWS provides exactly that.

QA ANALOGY

Compare owning your own load-generator machines (patching, capacity, maintenance) with using a cloud load-testing service where you just say "give me 500 virtual users". ECS on EC2 is the first; Fargate is the second.

	ECS (orchestrator)	Fargate (compute engine)
Role	Decides what runs, how many, deploys, replaces	Provides the CPU/memory each task runs on
Servers	With EC2 launch type: you manage instances	No servers to manage or patch
Billing	EC2: pay for instances even if idle	Pay per task vCPU + memory per second
Isolation	Tasks share instances	Each task isolated, with its own network interface

WHY FARGATE FOR THIS PROJECT

A small, stateless API with two tasks: no servers to patch, per-task security groups (because each task gets its own network interface in `awsvpc` mode), and scaling is just changing the desired count.

Interview one-liner: "ECS is the orchestrator; Fargate is the serverless compute it runs tasks on, so I don't manage EC2 instances and pay per task for the CPU and memory requested."

12 Amazon RDS

Relational Database Service

Database

RDS runs a **managed database** (here PostgreSQL). AWS handles installation, patching, backups and hardware; you just connect and use it.

QA ANALOGY

Your test environments probably had a shared test DB that everyone used. With two app servers, both must share **one** database – otherwise each server has its own data. You saw this bug live: in-memory storage made new employees "disappear" about half the time because the ALB alternated between two tasks with different data.

IN YOUR PROJECT

`employee-db`, PostgreSQL 18.3 on db.t4g.micro, 20 GB, encrypted, private (no public access), only reachable from `emp-ecs-sg`. The app connects with TLS and verifies the server certificate. The table is created automatically at app startup.

Things you met along the way

- **Aurora vs RDS**: Aurora Serverless v2 bills a minimum capacity even when idle – your first database would have cost ~$6/day. Always check the **minimum** setting of anything "serverless".
- **Stop/Start**: RDS can be stopped to save money (AWS restarts it after 7 days). Start the DB **before** the app, because the app connects at startup.
- **Subnet group**: the set of (private) subnets RDS may use.

What goes wrong

- Connection timeout → security group
- Authentication failed → wrong secret mapping
- Too many connections under load → pool size
- App crashes when DB restarts (unhandled error)

How to test it

- Write via task A, read via task B → same data
- Stop the DB → app returns 500, stays alive, recovers
- Load test → watch DB connections and CPU

Interview one-liner: "Data lives in a private, encrypted RDS PostgreSQL instance that only the ECS security group can reach; a shared database is what makes multiple stateless tasks correct."

13 AWS Secrets Manager (and KMS)

Secret storage

Security

Secrets Manager **stores passwords and keys securely** (encrypted with KMS), controls who can read them, and can rotate them. Your app never has a password written anywhere in code.

> **QA ANALOGY**
> Remember test scripts with hard-coded passwords in them, shared over email? This is the opposite: the password lives in a vault, and only an approved "badge" (the execution role) can open that one specific drawer.

How the password reaches your container

1. RDS created the master password and stored it in Secrets Manager (you never saw it).
2. The task definition contains only a **reference**: `DB_PASSWORD ← arn:…:secret:rds!db-…:password::`
3. When a task starts, the ECS agent uses `ecsTaskExecutionRole` to fetch the value.
4. ECS injects it as an environment variable inside the container. The app reads `process.env.DB_PASSWORD`.

Value vs ValueFrom: *Value* is plain text stored in the task definition (fine for a hostname). *ValueFrom* is a reference fetched at runtime (required for secrets). `:username::` means "take the `username` field of the JSON secret, current version".

KMS (Key Management Service) holds the encryption keys. Your secret uses the AWS-managed key, so no extra permission was needed.

Interview one-liner: "The DB password is RDS-managed in Secrets Manager; the task definition only holds the ARN, and the ECS agent fetches it at task start using an execution role scoped to that one secret."

14 AWS AppSync & GraphQL

Managed GraphQL API

API layer

AppSync hosts a **GraphQL API**. Clients send one query describing **exactly the fields they want**; AppSync checks authentication, validates the query against a **schema**, and runs **resolvers** that fetch the data – here by calling your REST API through the ALB.

> **QA ANALOGY**
> The **schema** is a contract – like an API spec you'd write contract tests against. AppSync rejects any query that doesn't match it before it reaches your backend, the way input validation rejects a bad request.

Term	Meaning	Your setup
Schema	Types and queries the API supports.	`Employee`, `employees`, `employee(id)`
Data source	Where data comes from (HTTP, DynamoDB, Lambda…).	HTTP → the ALB
Resolver	Code that maps a field to a backend request and shapes the response.	JavaScript: `GET /employees`, parse JSON
Authorization	Who may call the API.	API key (`x-api-key`)
Visibility	Public, or private (reachable only inside a VPC).	Public – the first one was private by mistake

Why AppSync sits between CloudFront and the backend

- A typed contract for clients; the backend can change behind it.
- Authentication before anything reaches your network.
- Can combine several backends into one response.
- Fully managed, with built-in logs and metrics.

What goes wrong (all happened to you)

- Private API → console shows "Cannot read properties of undefined (reading 'data')"
- Response body not parsed → wrong shape
- Resolver ignores the `id` argument

How to test it

- AppSync console → Queries
- No key / wrong key → 401
- Unknown id → `null`, not an error
- Enable logging to see resolver request/response

Interview one-liner: "AppSync gives a schema-validated GraphQL façade with API-key auth; an HTTP resolver translates the query into a REST call to the ALB and returns only the fields the client asked for."

15 Amazon CloudFront

Content Delivery Network (CDN)

Edge / entry point

CloudFront is a global network of **edge locations**. Users connect to the nearest one over HTTPS; CloudFront then forwards the request to your **origin** (AppSync) and can cache responses.

PERFORMANCE ANALOGY

You know network latency dominates response time for distant users. A CDN shortens the "first mile" by terminating HTTPS close to the user. For cacheable content it removes the backend call altogether – but GraphQL POST data is dynamic, so here caching is **disabled** on purpose.

Setting	What it does	Your choice
Origin	Where requests are forwarded	AppSync hostname, HTTPS only
Viewer protocol policy	HTTP handling for users	Redirect HTTP → HTTPS
Cache policy	What gets cached and for how long	CachingDisabled
Origin request policy	Which headers/query strings go to the origin	AllViewerExceptHostHeader

The Host header lesson

Forwarding *all* viewer headers sent `Host: …cloudfront.net` to AppSync, which routes by Host and answered 403. The fix was "all headers **except** Host". General rule: in front of AppSync, API Gateway or an HTTPS ALB, never forward the viewer's Host header.

The hidden auth bypass

Adding the API key as a CloudFront "custom origin header" made CloudFront authenticate every request – so anyone could query without a key. A good tester's instinct ("what happens with *no* key?") caught it.

Interview one-liner: "CloudFront is the public HTTPS entry point; caching is disabled for GraphQL POSTs, and it forwards everything except the Host header so AppSync can route the request and still see the API key."

16 IAM roles, STS & OIDC

Temporary, keyless access

Identity

Instead of storing AWS access keys in GitHub, your pipeline **proves who it is** with a signed token and receives **temporary credentials** (about one hour). Nothing long-lived exists to leak.

QA ANALOGY

Like a visitor pass: security checks your ID at reception (GitHub's signed token), confirms you're on today's approved list (the role's trust policy: this repo, main branch), and gives you a badge that expires tonight (STS credentials).

The flow

1. The GitHub Actions job (with `id-token: write`) asks GitHub for a signed OIDC token.
2. `configure-aws-credentials` calls **STS AssumeRoleWithWebIdentity** with that token.
3. AWS checks the **issuer** (identity provider), **audience** (`sts.amazonaws.com`) and **subject** (repo + branch) against the role's **trust policy**.
4. STS returns temporary credentials; the job pushes to ECR and deploys to ECS.

Two kinds of policy on a role

Policy	Answers	Your CI role
Trust policy	**Who** may assume the role	Only `AkashAjayInApp/employee-management-api`, branch `main`
Permissions policy	**What** the role may do	Push to one ECR repo, deploy to ECS, pass the execution role only to ECS

Execution role vs task role

Execution role – used by the ECS agent to pull the image, write logs and fetch secrets. **Task role** – used by your app code to call AWS APIs. Your app calls no AWS APIs, so it has no task role (least privilege).

> **The bug you solved**
>
> "Not authorized to perform sts:AssumeRoleWithWebIdentity". CloudTrail showed GitHub sent an *immutable* subject (`repo:Owner@id/repo@id:…`) while the trust policy expected `repo:Owner/repo:…`. Comparing "expected vs actual" – a tester's core skill – found it.

Interview one-liner: "GitHub Actions authenticates with OIDC: AWS validates the signed token against the role's trust policy and returns hour-long credentials, so no AWS keys are stored anywhere."

17 Amazon CloudWatch

Logs, metrics, dashboards, alarms

Observability

CloudWatch collects **logs** (what the app printed), **metrics** (numbers over time: CPU, requests, errors, latency), shows them on **dashboards**, and raises **alarms** when a metric crosses a threshold.

PERFORMANCE ANALOGY – THIS IS YOUR HOME GROUND

A CloudWatch dashboard is the live monitoring screen you watch during a load test. Alarms are automated SLA checks. The difference: in production it runs 24/7 and wakes someone up instead of waiting for you to look.

Feature	Your setup
Logs	`/ecs/employee-task`, one stream per task, kept 7 days
Metrics	ECS CPU/memory; ALB RequestCount, 5xx counts, TargetResponseTime, healthy hosts
Dashboard	`employee-api`: CPU & memory, requests & 5xx, latency p50/p95/p99, target health, alarm status, recent logs
Alarms	CPU > 80% (3 of 3 min), unhealthy targets ≥ 1, app 5xx > 5/min

Concepts worth knowing

- **Statistic**: Average, Sum, Minimum, Maximum, or percentiles (p95, p99). Use Sum for counts, percentiles for latency.
- **Period**: the bucket size (1 minute). **Datapoints to alarm** ("3 out of 3") avoids alerting on a single spike.
- **Missing data**: decide whether "no data" is good (no errors) or bad.
- **Alarm states**: OK, ALARM, INSUFFICIENT_DATA.
- **The golden signals**: latency, traffic, errors, saturation – your dashboard covers all four.

Interview one-liner: "CloudWatch gives me logs, metrics and alarms; my dashboard covers the four golden signals, and alarms on CPU, unhealthy targets and 5xx notify SNS."

18 Amazon SNS

Simple Notification Service

Notifications

SNS is a **message broadcaster**. Something publishes a message to a **topic**; every **subscriber** (email, SMS, Slack via Lambda, PagerDuty…) receives it.

QA ANALOGY

A distribution list for test-failure notifications – except the sender is an alarm, not a person.

IN YOUR PROJECT

Topic `employee-alerts`; email subscriber. All three alarms send on ALARM and on OK. **Lesson learned:** an email subscription stays "PendingConfirmation" until you click the link – until then alarms fire silently.

```
# Test the whole alert path without breaking anything
aws cloudwatch set-alarm-state --alarm-name employee-ecs-cpu-high \
    --state-value ALARM --state-reason "test" --region us-east-1
```

19 · AWS CloudTrail – the audit log

CloudTrail records **every API call made in your account**: who did what, when, from where, and whether it succeeded.

QA ANALOGY

The test-execution audit trail: "who changed this config, and when?" It's your evidence when something changed unexpectedly.

IN YOUR PROJECT

CloudTrail solved the pipeline failure: the failed `AssumeRoleWithWebIdentity` event showed the exact identity GitHub sent. It also showed when the role and its policy were created. Whenever AWS says "not authorized", look in CloudTrail for the denied call.

20 GitHub Actions (CI/CD)

Continuous Integration / Continuous Delivery

Automation

A **pipeline** runs automatically on every push: build, test, package, scan, deploy, verify. If any stage fails, the later stages don't run – broken code never reaches production.

> **QA ANALOGY**
> This is your regression suite promoted to a **gatekeeper**. Instead of "QA signs off", the pipeline signs off – every time, in minutes, with the evidence saved in the run log.

Stage	What it proves
Build (`npm ci`)	Dependencies install from the lock file
Unit test	9 tests pass – behaviour hasn't regressed
Docker build	The image builds (tag = commit SHA)
Trivy scan	No fixable HIGH/CRITICAL vulnerabilities
Push to ECR	Artefact stored and scanned again
Deploy to ECS	Rolling update completes and the service is stable
Verify	Right revision running, `/health` UP, GraphQL via CloudFront returns data – a **post-deployment smoke test**

Vocabulary

Term	Meaning
Workflow	The YAML file `.github/workflows/deploy.yml`
Job	A group of steps on one runner; jobs can depend on each other (`needs`)
Step / action	One command, or a reusable action like `actions/checkout@v4`
Runner	The temporary VM that executes the job (`ubuntu-latest`)
Variable / secret	Repository settings: `AWS_ROLE_ARN` (variable), `APPSYNC_API_KEY` (secret, masked)
Concurrency	Prevents two deployments running at once

> **Tester's tip: test the pipeline too**
> `actionlint` found a real bug in the verify step – `! grep` would never have failed the job. A check that can't fail is not a check. Always prove your gates can go red.

Interview one-liner: "Every push runs build, unit tests, Docker build, a Trivy security gate, push to ECR, a rolling ECS deploy and an automated smoke test; any failure stops the release."

21 Trivy & ECR scanning

Vulnerability scanning (DevSecOps)

Security testing

Scanners compare every package inside your image against databases of known vulnerabilities (**CVEs**) and report severity and the version that fixes it.

QA ANALOGY

Automated security regression testing. It doesn't find new bugs in your code; it finds **known** bugs in the components you ship – like checking that every third-party library is a supported, patched version.

WHAT IT FOUND IN YOUR PROJECT – AND THE FIX

Finding	Root cause	Fix
HIGH CVE in `zlib`	Base image older than the patched Alpine package	`RUN apk upgrade --no-cache`
10 HIGH CVEs in Node packages	All inside **npm's own** bundled packages, unused at runtime	Remove npm/corepack from the runtime image

Key flags: `--severity HIGH,CRITICAL` (what blocks), `--ignore-unfixed` (don't block on issues nobody can fix yet), `--exit-code 1` (fail the pipeline). This is the same idea as defining **exit criteria** for a test phase.

Interview one-liner: "Trivy runs in CI and fails the build on fixable HIGH/CRITICAL CVEs; ECR re-scans on push. It caught a zlib CVE and npm's bundled CVEs, which I fixed by patching the base and removing npm from the runtime image."

22 Terraform (Infrastructure as Code)

Infrastructure described in code

Automation – your next step

Terraform lets you **describe infrastructure in files** (`.tf`). Terraform compares the files with what exists in AWS and creates, changes or deletes resources to match. You built this project through the console (permitted by the brief), so Terraform is the most valuable thing to learn next.

> **QA ANALOGY**
> Manual infrastructure is like manual test execution: slow, error-prone and hard to repeat. Terraform is test **automation for environments** – the same reviewed script creates an identical environment every time. Your own mistakes (an env-var typo, a misnamed SNS topic, a wrong VPC selected) are exactly what IaC prevents.

The four commands

Command	What it does	QA equivalent
`terraform init`	Download providers, connect to the state backend	Set up the test framework
`terraform validate`	Check syntax and references	Lint / compile the test scripts
`terraform plan`	Show exactly what would change – nothing is changed yet	Dry run / review the test plan
`terraform apply`	Make the changes	Execute

State

Terraform keeps a **state file** mapping your code to real resources. It can contain sensitive values, so it is **never committed to Git** (your `.gitignore` already excludes it). Teams store it in an S3 bucket with a DynamoDB lock so two people can't apply at once.

A tiny example from your project

```
resource "aws_security_group" "ecs" {
  name   = "emp-ecs-sg"
  vpc_id = aws_vpc.main.id
}

resource "aws_vpc_security_group_ingress_rule" "ecs_from_alb" {
  security_group_id            = aws_security_group.ecs.id
  referenced_security_group_id = aws_security_group.alb.id   # only from the ALB SG
  from_port   = 8080
  to_port     = 8080
  ip_protocol = "tcp"
}
```

Suggested first exercise: write `security-groups.tf` for your three groups, then use `terraform import` (or import blocks) to bring the existing ones under Terraform and confirm `plan` shows "No changes".

23 Cost management

In DevOps, **cost is a quality attribute** – like performance. You learned this directly: an Aurora setting would have cost ~$170/month idle, and running two copies of the environment doubled the bill.

Resource	How it's billed	Can it be paused?
NAT Gateway	Per hour + per GB (≈ $1/day)	No – only deleted
ALB	Per hour + capacity units	No – only deleted
Fargate tasks	Per second of vCPU + memory	Yes – desired count 0
RDS	Per hour (instance) + storage	Yes – stop (up to 7 days)
Public IPv4 addresses	Per hour each	Removed with the resource
CloudFront, AppSync, SNS	Per request	Near zero when idle

Habits

- Set an **AWS Budget** alert (e.g. $20/month) on day one.
- Check "minimum capacity" on anything serverless.
- Delete old environments after a migration; set log retention.
- Use **Cost Explorer** (data lags about a day).
- Pause order: app first, then DB. Resume order: DB first, then app.

24 Hands-on exercises on your setup

Do these with your real system. Each is a small experiment: predict, act, observe, explain – exactly how you'd run a test. Restore anything you change.

Exercise 1 – Self-healing

Services: ECS, ALB · 10 minutes

Predict: what happens to traffic and to the task count when a task dies?

Do: keep a request loop running against the ALB, then ECS → service → Tasks → stop one task.

Observe: service Events tab, target group health, your loop's status codes.

Explain: running < desired → scheduler starts a replacement; ALB stopped routing to the stopped task.

Exercise 2 – Zero-downtime deployment

Services: GitHub Actions, ECR, ECS · 15 minutes

Do: change the message on `GET /`, push, and run a probe loop counting status codes during the deploy.

Observe: Deployments tab (old and new side by side), error count, the new message.

Explain: 100%/200% rolling update, health checks, deregistration delay, SIGTERM handling.

Exercise 3 – Break the network on purpose

Services: security groups, ALB, CloudWatch · 10 minutes

Do: temporarily remove the 8080 rule from `emp-ecs-sg`. **Restore it afterwards.**

Observe: targets turn unhealthy, ALB returns 503, the unhealthy-targets alarm fires and emails you.

Explain: health checks can't reach the tasks → no healthy targets → 503. This is your troubleshooting guide in action.

Exercise 4 – Load test and watch the dashboard

Services: ALB, ECS, CloudWatch · 20 minutes · your specialty

Do: run Locust (or JMeter) against `http://<alb>/employees`, ramping users over a few minutes. Keep it short.

Observe: CPU climbing, p95/p99 latency, RequestCount, the CPU alarm after 3 minutes above 80%.

Explain: find the throughput at which latency degrades → that's the capacity of 2 × 0.25 vCPU tasks → use it to set an auto-scaling target.

Exercise 5 – Database outage

Services: RDS, ECS, CloudWatch · 15 minutes

Do: stop the RDS instance while the app is running, then start it again.

Observe: `/employees` returns 500, `/health` stays 200, tasks are *not* replaced, the app recovers by itself.

Explain: why the liveness check deliberately ignores the database.

Exercise 6 – Prove the pipeline gate works

Services: GitHub Actions · 10 minutes

Do: on a branch, break a unit test (or temporarily lower Trivy's severity threshold) and push.

Observe: the pipeline goes red and later stages are skipped; nothing is deployed.

Explain: a quality gate is only real if it can fail.

25 Your performance-testing edge

Many DevOps engineers come from development or operations and have never designed a proper load test. You have. Use it deliberately:

DevOps decision	How your experience improves it
Task CPU/memory sizing	Find the knee of the latency curve under load, not a guess.
Auto-scaling targets	Pick the metric (CPU vs requests per target) and threshold from measured data; test scale-out speed vs ramp rate.
Alarm thresholds	Know normal variance, so alarms catch real problems without noise.
SLOs	Express targets as percentiles ("p95 < 300 ms") and error rates – the language you already use.
Deployment safety	Measure errors during deployments (you did: 0 of 200).
Database connections	Spot pool exhaustion under concurrency before production does.
Resilience	Run failure tests under load – killing a task at 70% capacity is very different from killing it when idle.
Performance in CI	Add a short smoke-load test stage (e.g. k6 for 60 s with p95 thresholds) to the pipeline as another gate.

> **Positioning statement**
>
> "I'm a DevOps engineer with a performance-engineering background: I don't just deploy systems, I prove how they behave under load and under failure, and I turn that into automated gates, alarms and scaling policies."

26 Glossary

Term	Meaning
ACM	AWS Certificate Manager – free TLS certificates for ALB/CloudFront.
ARN	Amazon Resource Name – unique identifier of an AWS resource.
Artefact	A build output (here a Docker image) that gets deployed.
AZ	Availability Zone – isolated data centre(s) within a region.
awsvpc	ECS network mode where each task gets its own network interface and IP.
Blue/green	Deploy the new version alongside the old, then switch traffic at once.
Circuit breaker	ECS feature that stops a failing deployment and rolls back.
CIDR	Notation for IP ranges, e.g. 10.0.0.0/16.
CI / CD	Continuous Integration (build + test every change) / Continuous Delivery or Deployment (release automatically).
CVE	A publicly known, numbered security vulnerability.
Desired count	How many tasks an ECS service should keep running.
Drift	Real infrastructure differing from its definition (code or docs).
Edge location	CloudFront point of presence close to users.
Golden signals	Latency, traffic, errors, saturation.
Graceful shutdown	Finishing in-flight work before exiting on SIGTERM.
Idempotent	Running it twice gives the same result (Terraform apply, PUT).
IaC	Infrastructure as Code (Terraform, CloudFormation).
Immutable tag	An image tag that always points to the same image (version, commit SHA).
Least privilege	Granting only the minimum permissions needed.
Liveness vs readiness	"Is the process alive?" vs "Is it ready to serve traffic (dependencies OK)?"
NAT	Network Address Translation – outbound-only internet for private subnets.
OIDC	OpenID Connect – token-based identity standard used for keyless CI access.
Origin	The backend CloudFront forwards to (AppSync here).
Resolver	AppSync code connecting a GraphQL field to a data source.
Rolling update	Replace instances gradually while keeping capacity.
Runner	Machine that executes CI jobs.
SHA	Git commit identifier; used as an image tag for traceability.
SIGTERM	The "please stop" signal ECS sends before killing a container.
SLO / SLA	Service level objective (internal target) / agreement (contractual promise).
Stateless	App keeps no data in memory between requests – required to run many copies.

STS	Security Token Service – issues temporary AWS credentials.
Target group	ALB's pool of backends plus health-check settings.
Task definition	Versioned ECS blueprint for running a container.
Trust policy	The part of an IAM role that says who may assume it.

27 Learning path: next 90 days

When	Focus	Concrete goals using this project
Weeks 1–2	Consolidate	Re-explain each service from this guide out loud without notes. Do Exercises 1–6. Write the repository README in your own words.
Weeks 3–5	Terraform	Codify the project in the assessment's file layout (`vpc.tf` , `security-groups.tf` , `ecr.tf` , `ecs.tf` , `alb.tf` , `appsync.tf` , `cloudfront.tf` , `iam.tf` , `monitoring.tf`). Remote state in S3. Add `terraform plan` to CI on pull requests.
Weeks 6–7	Security hardening	HTTPS on the ALB (ACM + domain), block direct ALB access with a secret header rule, WAF rate limiting on CloudFront, Cognito instead of API keys.
Weeks 8–9	Scaling & performance	ECS auto scaling; load-test it; add a k6 smoke-load stage to the pipeline with p95 thresholds.
Weeks 10–11	Observability	Structured JSON logs with request IDs, Container Insights, X-Ray tracing across AppSync → ALB → ECS.
Week 12+	Broaden	Kubernetes basics (EKS concepts map to ECS: pod ≈ task, deployment ≈ service), and the AWS Certified Cloud Practitioner → Solutions Architect Associate / DevOps Engineer certifications.

Final thought

You already think like a reliability engineer: you question assumptions, test the unhappy path and demand evidence. In this project that instinct found an authentication bypass, a crash on database restart, a permission bug and a pipeline check that could never fail. That is the value you bring to a DevOps team.